

论文解构报告

Towards High-Goodput LLM Serving with Pre-fill-decode Multiplexing

Yukang Chen, Weihao Cui, Han Zhao, Ziyi Xu, Xiaoze Fan, Xusheng Chen, Yangjie Zhou, Shixuan Sun, Bingsheng He, Quan Chen

ASPLOS '26 (31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2) · 2026

LLM Serving

PD-Multiplexing

Goodput

GPU 调度

SLO

源文件：本地上传文件（无外部链接）

DOI: <https://doi.org/10.1145/3779212.3790236>

生成时间：2026-07-29

目录

摘要翻译	3
1. 一句话总览	3
2. 阅读范围与证据状态	3
3. 根问题：论文究竟要解决什么	3
4. 旧方法为何不够	5
5. 核心洞见与假设	6
6. 方法：从洞见到可执行系统	7
7. 为什么它可能有效	9
8. 实验与指标：证据是否支撑主张	9
9. 图表解读与论证关系	11
10. 完整因果推理树	13
11. 结论、贡献与边界	13
12. 术语与第一性原理学习路径	14

摘要翻译

大语言模型 (LLM) 服务需要同时满足预填充 (prefill) 和解码 (decode) 阶段严格的服务级别目标 (SLO)。现有方案中, 一些将两个阶段分离部署, 可能导致资源闲置或计算冗余; 另一些将预填充阶段切分为多个块并与解码迭代融合, 造成 SLO 合规性与高利用率之间的两难困境。为解决这些问题, 高效的服务系统应能动态调整计算资源分配、将计算与内存管理解耦, 并使预填充与解码独立执行。我们提出了 MuxWise, 一个采用全新范式——GPU 内预填充-解码复用 (intra-GPU prefill-decode multiplexing)——的 LLM 服务框架。为充分发挥该范式的优势, MuxWise 集成了无气泡复用引擎、抗争用估计器和 SLO 感知调度器。评估结果表明, 在满足 SLO 保证的前提下, MUXWISE 相较于现有最先进方案将峰值吞吐量平均提升了 2.20 倍 (最高达 3.06 倍)。

1. 一句话总览

论文元数据

字段	内容
标题	Towards High-Goodput LLM Serving with Prefill-decode Multiplexing
作者	Yukang Chen, Weihao Cui, Han Zhao, Ziyi Xu, Xiaoze Fan, Xusheng Chen, Yangjie Zhou, Shixuan Sun, Bingsheng He, Quan Chen
发表场所 / 年份	ASPLOS '26 (第 31 届 ACM 架构支持编程语言与操作系统国际会议, 第 2 卷) / 2026
研究领域	计算机系统 (LLM 服务系统)
关键词	LLM Serving, PD-Multiplexing, Goodput, GPU 调度, SLO

资源链接

- 原始文件: 本地上传文件 (无外部链接)
- arXiv: 文中未说明
- DOI: <https://doi.org/10.1145/3779212.3790236>
- 代码 / 数据集: 文中未说明

标签 (5 个) : LLM 推理服务 · Prefill-Decode 多路复用 · GPU 调度 · SLO 保障 · Goodput 优化

论文提出 MuxWise, 一种让 prefill (预填充, 处理输入生成首 token) 与 decode (解码, 逐 token 生成) 在同一块 GPU 内部、同一时刻分别占用不同计算单元并行执行的新范式, 配合无气泡多路复用引擎、抗争用估计器、SLO 感知调度器三个模块, 在满足延迟 SLO 前提下将有效吞吐 (goodput) 平均提升 2.20 倍、最高 3.06 倍 [作者明确陈述]。

2. 阅读范围与证据状态

- OCR 覆盖论文正文第 1-15 页的文本证据分析与 6 组图表证据分析 (共 12 张图片, 另有 6 张因超出上限未做视觉分析), 可核实内容包括方法设计、消融数据、比较工作表与主张证据表。
- 明显缺失: Goodput/TTFT/TBT 的精确计算公式、统计聚合口径 (是否多次运行平均、置信区间) 文中未说明; 代码/数据集链接文中未说明。
- 6 张未分析图片可能包含额外实验细节, 本报告不对其做视觉层面判断, 相关结论只依据已分析的图组与文本。

3. 根问题: 论文究竟要解决什么

- **问题定义**: LLM 在线服务需同时满足两条独立延迟 SLO——prefill 阶段的首 token 时间 (TTFT, 如<500ms) 与 decode 阶段的逐 token 间隔 (TBT, 如<100ms) [作者明确陈述]。由于 prefill 与 decode 在同一请求流中交替出现, 长 prefill 会挤占正在进行的 decode 迭代, 造成 TBT 超标[作者明确陈述]。
- **为何重要**: 真实工作负载 (单轮聊天、长文档、长链推理、多轮对话/工具调用) 在输入/输出/复用 (reused, 跨轮共享上文) 长度上差异巨大 (Conversation 复用长度可达 0-120K token) [作者明确陈述], 任何静态资源分配都难兼顾吞吐与 SLO。
- **难点来源**: token 是基本工作单元, prefill 计算量随 (新增+复用) token 数增长、decode 显存访问量随复用长度增长; 若两阶段共享同一批资源却按同一批次串行调度, 一方资源需求波动即传导为另一方排队延迟, 最终表现为 SLO 违约与 goodput 下降 [基于文中逻辑的推断]。
- **本文 story**: 旧路线 (分离式部署、分块预填充) 都把 compute 与 memory 管理绑定在一起、且让两阶段在时间上交替, 这一结构性设计导致资源闲置或"SLO 合规 vs 高利用率"两难; 本文的转折在于把两阶段从"时间交替"改为"空间并行", 从而在同一 GPU 内解耦资源分配。

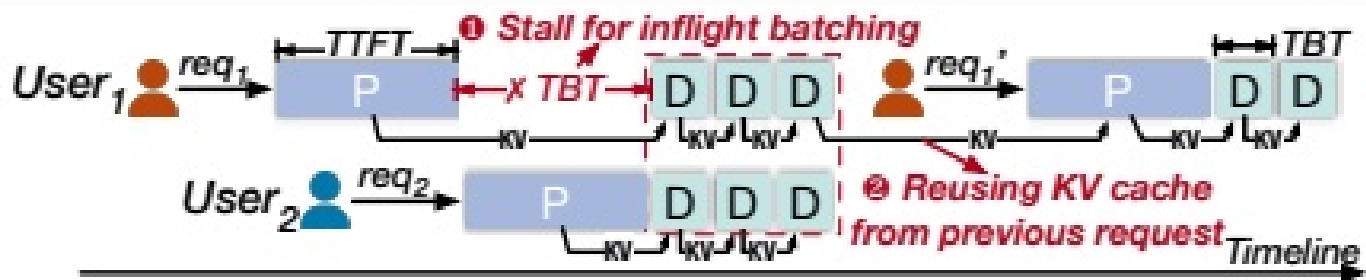


Figure 1. A typical workflow in LLM serving systems: User1 sends two consecutive requests, with the second reusing the context from the first. User2 sends 1 request.

图组解读

- **图组结论**：长 prefill 插入 decode 序列导致相邻 decode 间隔违反 TBT SLO，直观呈现根问题现象。
- **论证关系**：作为直观问题证据，支撑根问题中长 prefill 挤占 decode 导致 TBT 超标的论点，与定量分析互补。
- **适用范围**：为概念性工作流示意图，而非特定硬件或工作负载下的实测数据。

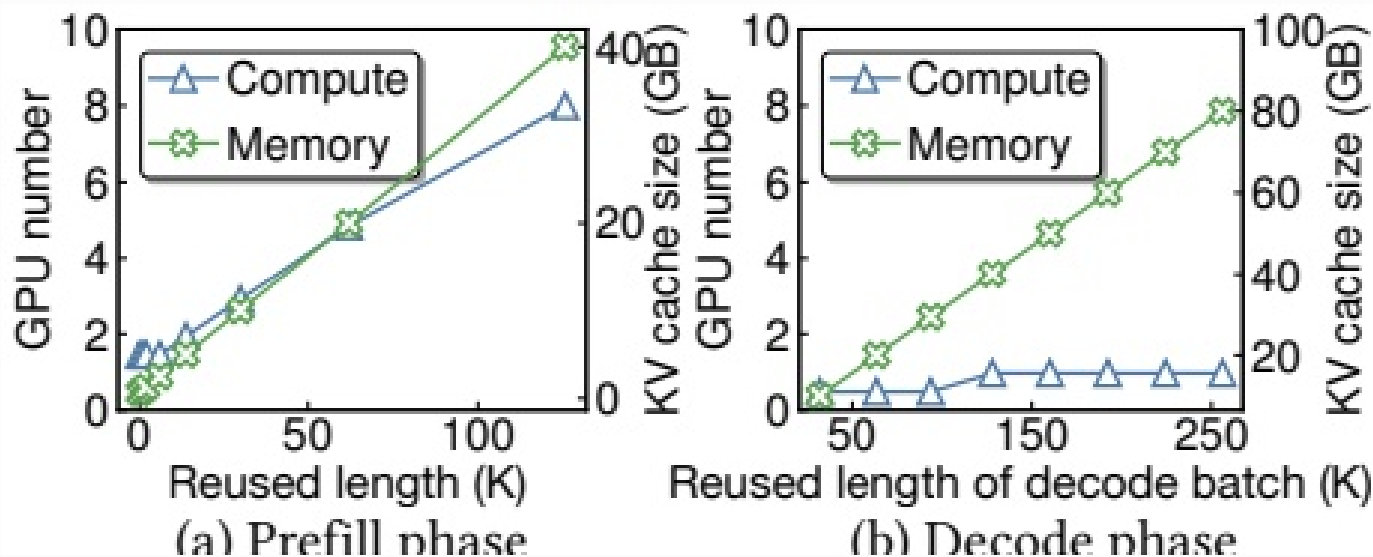
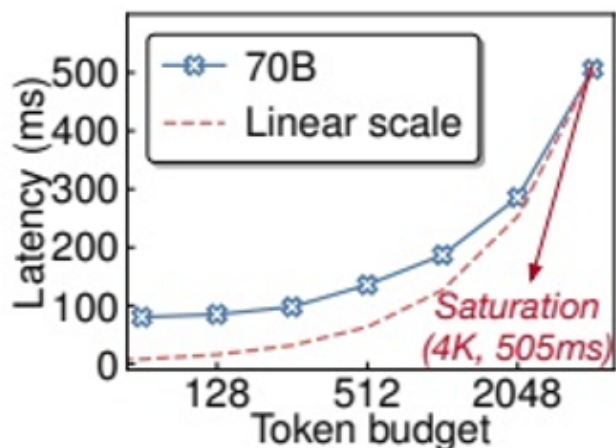


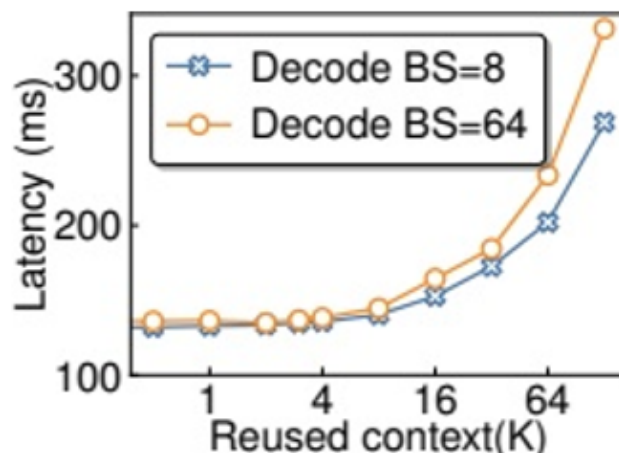
Figure 3. Required compute and memory for processing different phases under SLO constraints with varied reused context lengths. For prefill (a), the batch size is fixed at 1, the new context length is set to 2K, and TTFT is set to 400ms. For decode (b), the batch size is fixed at 32, and TBT is set to 100ms. These settings are commonly seen in online serving.

图组解读

- **图组结论**：Prefill 计算需求随复用长度显著上升，Decode 计算需求不敏感，而两阶段显存需求均持续增长。
- **论证关系**：定量揭示 prefill 与 decode 算力需求错配，支撑需要动态划分计算资源并共享显存的核心洞见。
- **适用范围**：限定于 Llama-70B、8×A100、特定批大小与 SLO 设置下的测量结果。



Panel 1



Panel 2

Figure 6. (a) Sweet spot of the token budget in chunk-prefill. The decode uses a fixed batch size of 32, with each request having a reused context length of 1K tokens. (b) Latencies with varied reused context of the fused prefill chunk in chunk-prefill. The token budget is fixed at 512, and the reused context length of decode phase is the same as in (a).

图组解读

- **图组结论**: token 预算达 4K 时延迟超 500ms 远超 SLO, 且 TBT 随复用长度增长显著恶化。
- **论证关系**: 证明分块预填充的利用率与延迟两难是结构性的, 构成空间复用范式的核心动机。
- **适用范围**: 仅覆盖 Llama3-70B 与 8×A100 配置, 不含 MuxWise 对照数据。

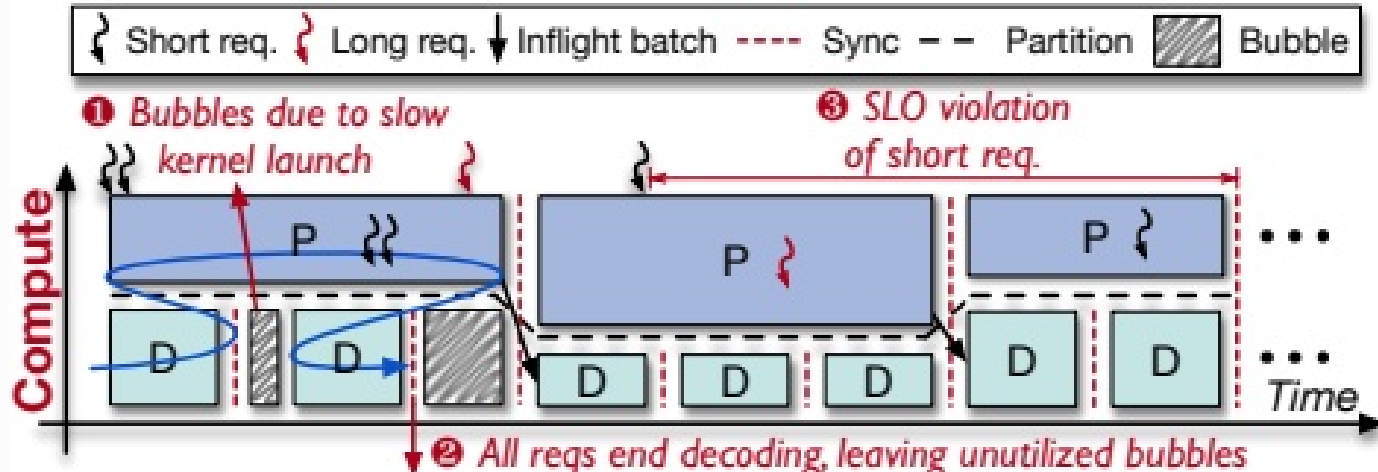


Figure 9. Bubbles and SLO violation of naive intra-process multiplexing. represents the kernel launch order.

图组解读

- **图组结论**: 朴素 GPU 内多路复用会引入启动开销气泡、资源无法回收气泡及短请求 SLO 违约。
- **论证关系**: 揭示朴素空间复用缺陷, 论证无气泡多路复用引擎层级调度与事件同步的必要性。
- **适用范围**: 为朴素调度缺陷的示意图, 无具体数值刻度。

4. 旧方法为何不够

Splitwise (静态分离式)

前提: 目标是把 prefill/decode 分离到独立实例分别保 SLO; 前提是负载可预先划分。

本文指出的失效点



本文对应模块
共享显存池设计

GPU 划分在实例初始化时固定，负载波动时产生闲置[作者明确陈述]；各实例独立缓存池使总容量缩水，命中率下降（引用文献数据 36.6%→4.2%）[作者明确陈述，引用[45]]。

检验：Figure 5 自测命中率曲线、§ 4 端到端对比

LoongServe（动态分离式）

前提：在 Splitwise 基础上按序列长度/阶段动态调整 GPU 分配。前提是可以随时释放原 GPU 缓存。

本文指出的失效点

为避免缓存重复而立即释放，导致跨请求（多轮）KV 复用失效，必须重算[作者明确陈述]。



本文对应模块

PD multiplexing 的共享显存设计

检验：Figure 4(b) 示意分析、Table 4 端到端对比

Chunked-prefill（如 SARATHI-Serve）

前提：把 prefill 切块与 decode 迭代融合，靠 token 预算上限保 SLO。前提是可找到兼顾利用率与延迟的预算值。

本文指出的失效点

70B 模型在 8×A100 打满 GPU 需 4K 预算，是 100ms TBT SLO 所允许 256 预算的 8 倍[作者明确陈述]；复用上下文越长，TBT 越差[作者明确陈述]。



本文对应模块

无气泡多路复用引擎（空间复用替代时间融合）

检验：Figure 6(a)(b) 预算/复用长度扫描

NanoFlow

前提：算子级 GPU 内多路复用，目标是宽松 SLO 下近最优吞吐。

本文指出的失效点

严格 SLO（≤100ms）要求预算降至 256 时不再计算受限，且 70B 模型因反复加载权重开销被放大 [作者明确陈述]。



本文对应模块

PD multiplexing 使模型权重仅 prefill 阶段加载一次

检验：Table 3/4、Figure 14(c)(d)

SGLang-PD

前提：静态分离但支持跨阶段缓存共享。

本文指出的失效点

波动负载下 decode 实例仍会闲置，OpenThoughts/LooGLE 场景下表现明显更差[作者明确陈述]。



本文对应模块

动态计算重配置能力

检验：§ 4.2.1、§ 4.3 端到端对比

DistServe、Dynamo

本文指出的失效点

仅有排除性文字说明，未作为主要基线，原始动机与限制无法从当前 OCR 内容确认。



本文对应模块

文中未作为主要基线对比

5. 核心洞见与假设

- **核心洞见**：与其让 prefill、decode 在同一流水线中"时间交替"（无论跨实例还是同 GPU 前后执行），不如让二者在同一 GPU 内部、同一时刻通过划分 SM（流式多处理器）"空间并行"执行，同时共享同一块显存 [作者明确陈述]。
- **实现组件(非独立贡献)**：GreenContext 空间划分（来自已有文献）、层级化 prefill 切分、query-based synchronization、离线画像预测器与 contention guard。
- **支撑前提**（[实验支持]）：prefill 计算需求对复用长度高度敏感、decode 相对不敏感（Figure 3a/b）；两阶段 KV 缓存量可达数十至数百 GB，宜共享显存空间。
- **前提可能失效处**：该范式依赖硬件支持进程内低开销空间划分（如 GreenContext），不支持该特性的硬件不适用[作者明确陈述]。
- 由此产生两个新挑战：C-1 朴素调度产生 GPU 气泡；C-2 空间复用引入不可预测的显存带宽争用 [作者明确陈述]。

6. 方法：从洞见到可执行系统

总览: 请求进入后, 估计器基于离线画像给出最坏情况延迟预测→调度器据此决定 SM 划分与请求调度顺序→引擎按层级化方式执行 prefill 并与 decode 并行→运行时数据反哺争用防护模块, 形成闭环 [作者明确陈述]。

- 1. **Bubble-less Multiplex Engine (无气泡多路复用引擎)**：动机是朴素调度会因 prefill 启动耗时 (数十 ms) 远超单次 decode 迭代 (<0.5ms) 产生气泡、GPU 不可抢占导致资源无法回收、长请求排队使短请求违约。做法: 把 prefill 按 transformer 层切分为"prefill layers (PLs)", 以接近 decode 迭代的粒度交替启动, 配合事件轮询同步在最后一层完成时立即合并进 decode batch。优势: 层级切分开销可忽略, 因 LLM 天然由多层组成; 带来抢占短请求优先的能力。
- 2. **Contention-tolerant Estimator (抗争用估计器)**：动机是 GreenContext 无法管理显存带宽争用、GPU 不暴露实时带宽监控接口。输入为复用/新增/输出长度、batch size、SM 分区; 过程分 solo-run predictor (拟合 $T_{Prefill}$ 、 T_{Decode} 线性模型, Eq.1/Eq.2) 与 contention guard (离线网格画像给出最坏减速因子)。输出为最坏情况延迟预测, 用于保证 SLO 上界而非追求精度。
- 3. **SLO-aware Dispatcher (SLO 感知调度器)**：动机是需据估计结果实际分配 SM 并选调度对象。每次 decode 迭代结束时重新决策, 优先保 decode SLO, 用 $N_{PL} = \lceil (T_d \times N_T) / T_P \rceil$ 计算应启动的 prefill 层数, 剩余 SM 给 prefill; 允许短请求抢占长请求但不允许递归抢占。

前排论文大图 方法流水线 呼应：第 6 节：具体主张

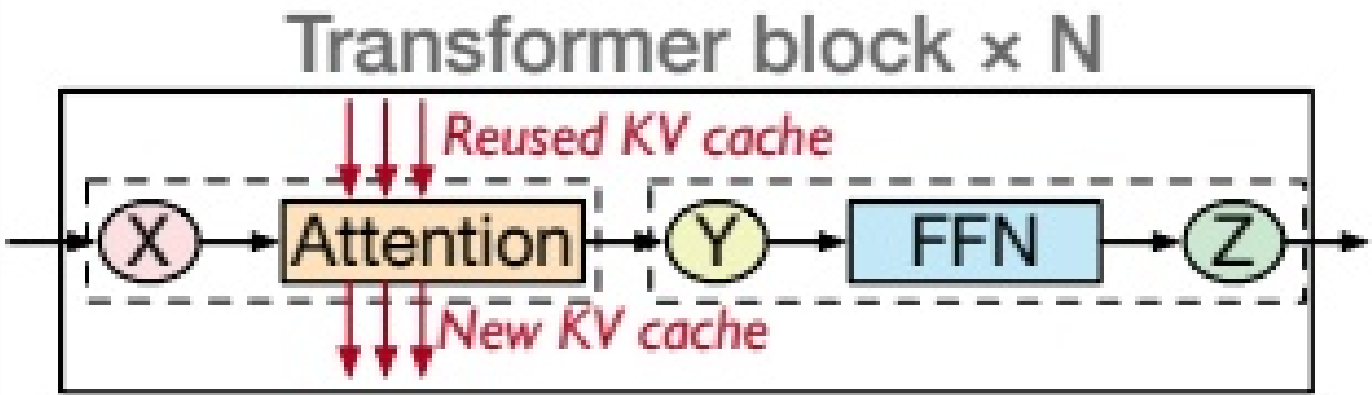


Figure 2. Main architecture of most LLMs.

图组解读

- **图组结论**: LLM 由多个相同 transformer 层堆叠而成, KV 缓存交互仅发生在 Attention 子模块。
- **论证关系**: 为无气泡引擎的按层切分 prefill 设计提供结构前提, 证明层级切分开销极低。
- **适用范围**: 主流 Transformer 架构 LLM 的通用结构抽象。

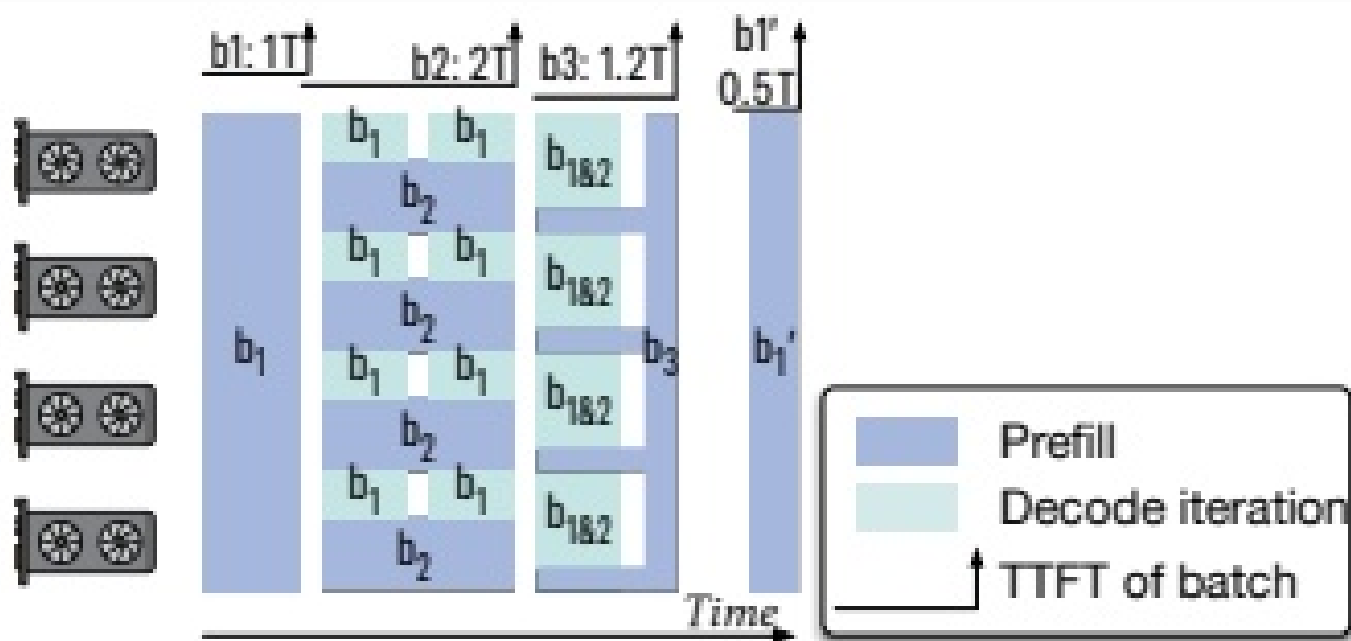


Figure 7. An ideal solution: prefill-decode multiplexing.

图组解读

- **图组结论：** Prefill 与 Decode 色块在同一时刻分布于不同资源行，实现 GPU 内空间并行执行。
- **论证关系：** 直观展示 PD 空间多路复用范式，说明两阶段解耦与计算资源动态共享机制。
- **适用范围：** 新范式概念示意图，图中数值为相对时间单位。

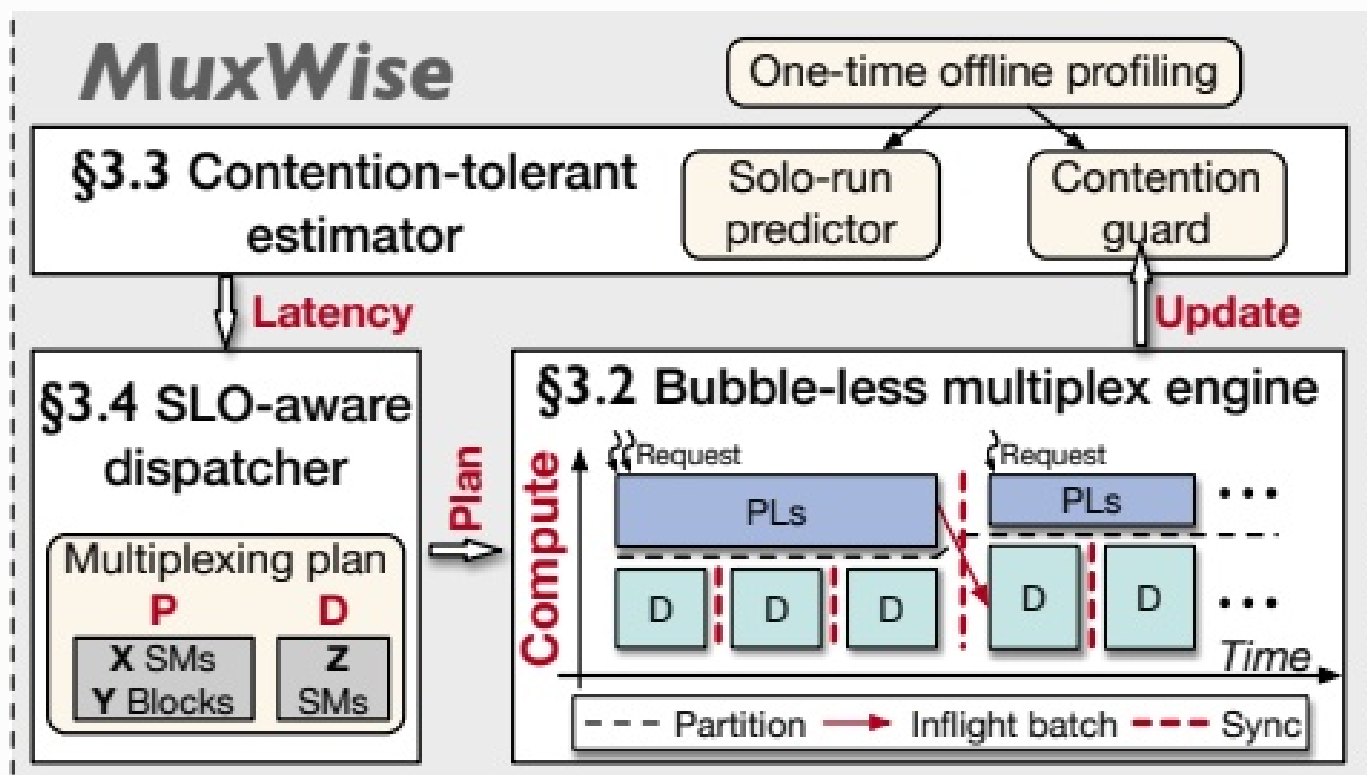


Figure 8. Architecture overview of MuxWise.

图组解读

- **图组结论：** 展示抗争用估计器、SLO 感知调度器与无气泡多路复用引擎三模块闭环协同关系。
- **论证关系：** 说明系统整体架构与数据流向，展示三模块如何协作解决气泡与争用挑战。
- **适用范围：** MuxWise 系统整体架构框图。

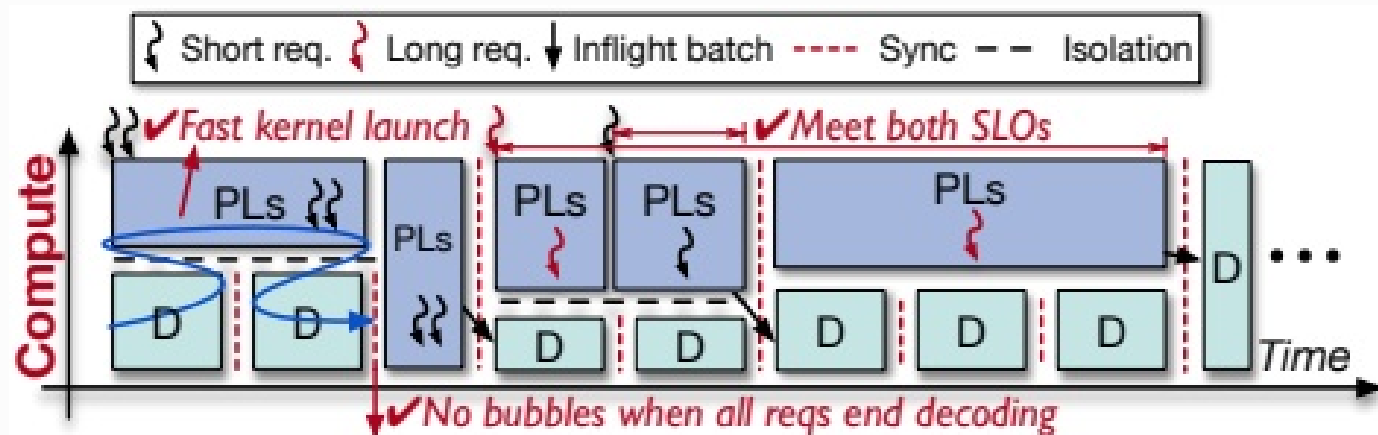


Figure 10. Bubble-less coordination using layer-wise scheduling for the prefill phase and graph-level scheduling for the decode phase. PLs is the short for prefill layers.

图组解读

- **图组结论：**通过层级化 prefill 切分与基于事件轮询的同步机制，实现无气泡的跨阶段协调。
- **论证关系：**展示无气泡引擎如何消除启动延迟、资源不可回收及短请求排队三类气泡。
- **适用范围：**无气泡协调机制的时序示意图。

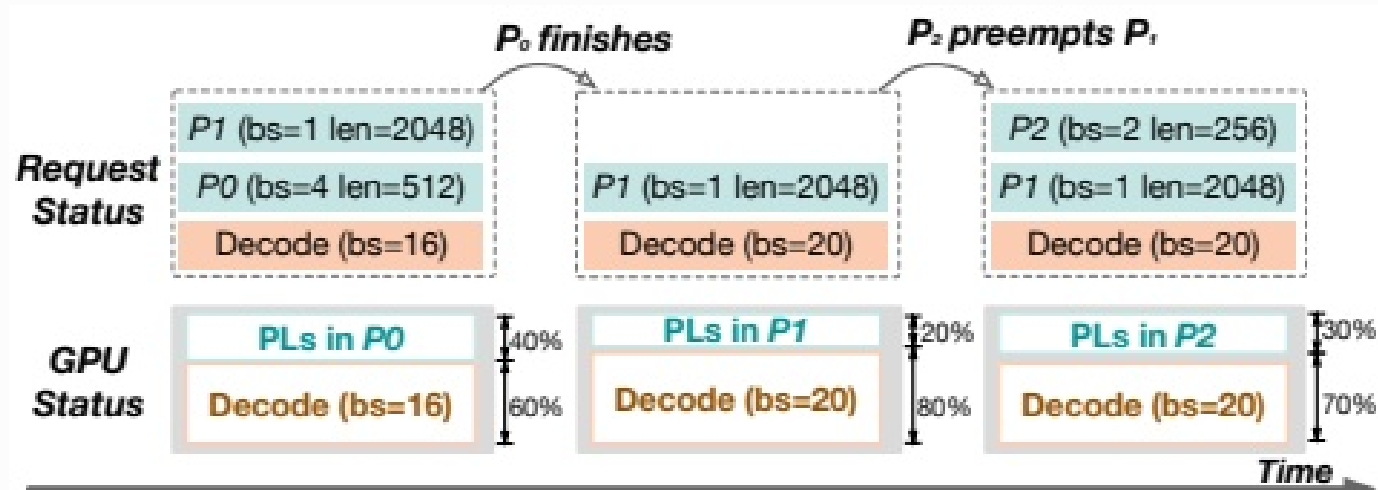


Figure 12. The dispatching policy of MuxWise.

图组解读

- **图组结论：**SM 划分比例随 decode 批次组成动态重算，且展示短请求抢占长请求调度过程。
- **论证关系：**直观呈现 SLO 感知调度器的分配逻辑与抢占机制，支撑调度策略实现细节。
- **适用范围：**调度策略流程图解，非实测运行记录。

7. 为什么它可能有效

- 层级化切分使 prefill 粒度接近 decode 迭代，从而快速让出资源、减少气泡 [基于文中逻辑的推断]，需要的前提是 LLM 确实由多层构成（普遍成立）。
- 共享显存空间保留 KV 缓存池的完整效率，前提是硬件显存容量足够容纳 prefill+decode 合并后的缓存需求 [基于文中逻辑的推断]。
- 空间并行使两阶段互不阻塞独立执行，从而理论上避免"SLO 合规 vs 高利用率"两难，前提是 SM 划分能被低开销动态调整（依赖 GreenContext）[基于文中逻辑的推断]。
- worst-case 延迟估计保证 SLO 上界，前提是离线画像覆盖范围包含线上实际出现的资源配置组合，论文未证明该覆盖的完备性 [基于文中逻辑的推断]。

8. 实验与指标：证据是否支撑主张

8.1 实验问题与判定标准

- 效果问题: MuxWise 是否在真实/合成负载下相对四个基线提升 TTFT/TBT/goodput? 对应 Figure 14-17 端到端对比[实验支持]。
- 必要性问题: chunked-prefill 的 token 预算两难是否架构性不可规避? 对应 Figure 6 扫描实验[实验支持]。
- 机制/必要性问题: 无气泡引擎的层级调度与同步机制是否必要? 对应 § 4.4.2 消融[实验支持]。
- 鲁棒性问题: 抢占机制是否带来提升? 对应 Figure 20, 仅有量化的"开启抢占"对比, 未提供"关闭抢占仍有提升"的量化数据[作者明确陈述, 但未见量化数据]。
- 适用范围问题: MoE 大模型 (Qwen3-235B) 上是否同样有效? 仅与 chunked-prefill 单一基线对比, 范围窄于其他实验[作者明确陈述]。

8.2 数据、任务、对比与运行设置

- 负载来源: ShareGPT (单轮聊天)、LooGLE (长文档)、OpenThoughts (长链推理)、Conversation/Tool&agent (多轮, Table 1 定义) [作者明确陈述]。
- 模型/硬件: Llama-8B/70B (A100/H100)、Qwen3-235B (H200), 均用张量并行部署[作者明确陈述]。
- 基线: chunked-prefill(SARATHI-Serve)、NanoFlow、LoongServe、SGLang-PD, 另有自建对比 WindServe、Tropical、Semi-PD、Bullet。
- 请求到达: Poisson process 模拟[作者明确陈述]。
- 重复次数/统计方式: 文中未说明。
- SLO 目标: Llama-8B 设 50ms TBT, Llama-70B 设 100ms TBT[作者明确陈述]。

8.3 指标字典与对应公式

- **指标与方向**: TTFT, 测量 prefill 耗时, 单位 ms/s, 越低越好。 **定义与公式**: 来源层级为无可核验公式; 文中仅文字定义为"产生首 token 的耗时"。 **实验用途**: SLO 达标判据, 对比 MuxWise 与四基线。 **读数与边界**: 具体倍数见 8.4; 不能证明 goodput 绝对值。
- **指标与方向**: TBT, 测量 decode 单 token 间隔, 单位 ms, 越低越好, 是全文核心 SLO 指标。 **定义与公式**: 来源层级无可核验公式; 文字定义为"每 token 延迟", 并与 TPOT (平均指标) 对比强调其为"逐 token"口径。 **实验用途**: 主 SLO 判据, Llama-8B 设 50ms、Llama-70B 设 100ms。 **读数与边界**: 不能反映平均表现掩盖的尾部延迟以外信息。
- **指标与方向**: Goodput (有效吞吐), 单位文中未说明 (推断 token/s 或 req/s), 越高越好。 **定义与公式**: 来源层级为推导关系 (非原文公式) [基于文中逻辑的推断: 推理依据——作者陈述"under the constraint of meeting the 99%-ile SLO guarantees"+报告吞吐倍数→goodput=SLO 达标条件下的吞吐→用于全文倍数对比], 非原文 Eq.。 **实验用途**: 摘要 2.20×/3.06×结论来源。 **读数与边界**: 分母口径为"同一实验设置下基线对应数值"[作者明确陈述], 但聚合方式 (多次运行平均/置信区间) 文中未说明。
- **指标与方向**: $T_{Prefill}$ 、 T_{Decode} , 估计器内部延迟预测量。 **定义与公式**: 来源层级为原文公式。 $T_{Prefill} = \theta_1 \cdot \sum_i^{bs} n_i^2 + \theta_2 \cdot \sum_i^{bs} n_i \cdot r_i + \theta_3 \cdot \sum_i^{bs} n_i + \theta_4$ (Eq.1); $T_{Decode} = \theta_1 \cdot \sum_i^{bs} r_i + \theta_2 \cdot bs + \theta_3$ (Eq.2), bs 为批大小, n_i/r_i 为第 i 请求新增/复用 token 数。 **实验用途**: 支撑 solo-run predictor 精度主张 (最大偏差 prefill 8.16%、decode 8.84%) [作者明确陈述]。
- **指标与方向**: 气泡率, CUDA stream 空闲时间占比, 越低越好 (论文认为不直接等价于 goodput 损失)。 **定义与公式**: 作者文字定义"the proportion of all such bubbles in the compute stream"。 **读数与边界**: MuxWise 气泡率 7.7% 高于 chunked-prefill 的 4.5%, 但作者称不影响 goodput, 该因果解释未见独立对照实验量化 [基于文中逻辑的推断]。

8.4 主结果、消融与证据边界 比较工作与被批判限制的呼应

比较对象	核心限制/失效条件	本文对应模块	直接实验与仍未验证边界
Splitwise	静态分配致闲置; 缓存池减半降命中率	共享显存池+动态引擎	Figure 5 命中率实验; § 4 端到端对比; 36.6%→4.2% 数字来自外部文献非本文自测
LoongServe	释放缓存致跨请求复用失效、需重算	PD multiplexing 共享显存	Figure 4(b)示意、Table 4 对比; 未给出重算量化占比
Chunked-prefill	token 预算大小两难, 复用越长 TBT 越差	无气泡多路复用引擎	Figure 6 扫描实验, 证据较充分
NanoFlow	严格 SLO 下非计算受限、70B 权重反复加载开销大	PD multiplexing 权重仅加载一次	Table 3/4、Figure 14(c)(d)
SGLang-PD	静态分配波动负载下闲置	动态计算重配置	§ 4.2.1、§ 4.3 对比; TBT 上 SGLang-PD 有时优于 MuxWise

- 端到端 TTFT 加速比 (Figure 14): 相对 chunked-prefill/NanoFlow/LoongServe/SGLang-PD 分别 3.57×、5.98×、4.65×、1.66× (排除不稳定结果后) [实验支持]。
- SLO 达标下 goodput (Figure 15): Llama-8B 为 2.6×/5.2×/2.0×/1.3×; Llama-70B 为 3.06×/2.62×/1.62× (对 chunked-prefill/LoongServe/SGLang-PD, 无 NanoFlow 数据) [实验支持]。
- 消融——无气泡引擎: 禁用层级调度使 TTFT 增约 10ms; 进一步禁用同步机制使 TBT 恶化 314ms (8B) /672ms (70B) [实验支持]。

- 消融——抢占调度：99%-ile TTFT per token 加速 $1.96\times$ [实验支持]；关闭抢占仍有提升的说法未给出量化数据[无法从当前 OCR 内容确认]。
- Qwen3-235B (MoE) 场景仅与 chunked-prefill 对比, LoongServe/分离式方案因不支持 MoE 被排除[作者明确陈述该限制]。

9. 图表解读与论证关系

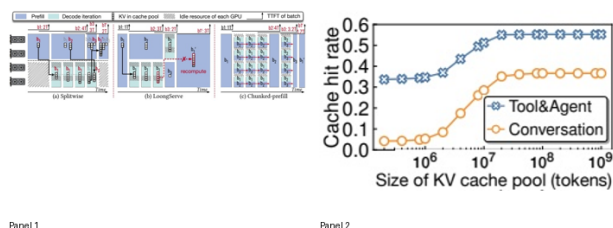


Figure 4. Processing four LLM request batches on 4 GPUs using (a) Splitwise, (b) LoongServe (c) chunked-prefill. All methods satisfy the TBT SLO (T per decode iteration). Specifically, arrives at 0T, at 1T, at 3T, and at 5T. denotes a subsequent request batch that reuses the KV cache of . Inefficient TTFTs are marked in red for each method. KV cache management is shown only for the two disaggregated methods, as they require migration or recomputation. In (a) and (b), solid black arrows represent migration, while dashed red arrows with cross markers denote recomputation. In (a), the KV cache column with a red indicates the active batch. In (c), the red arrow denotes KV cache reads from earlier chunks.

图组解读 **对比基线** 呼应：第 9 节：具体主张

- **图组结论**：Splitwise 存在 decode 实例闲置, LoongServe 跨请求复用产生重算, Figure 5 显示池越小命中率越低。
- **论证关系**：支撑分离式部署资源耦合致闲置与重算的批判, 自测命中率实验补充容量缩水危害。
- **适用范围**：示意图覆盖 4 卡 4 批次场景；命中率实验覆盖 Tool&Agent 与 Conversation 负载。

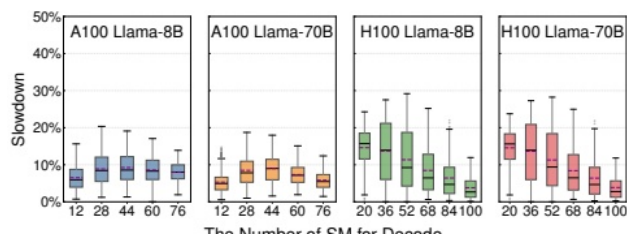


Figure 11. Slowdowns in decode due to contention with different multiplexing configurations of pre-fill and decode across models and GPUs.

图组解读 **消融验证** 呼应：第 9 节：具体主张

- **图组结论**：不同模型与 GPU 下 decode 因争用减速在 0%–30% 波动，且不随 SM 分配呈现单调规律。
- **论证关系**：证实显存带宽争用具有不可预测性，支撑抗争用估计器给出最坏情况减速上界的设计选择。
- **适用范围**：覆盖 Llama-8B/70B 与 A100/H100，仅展示 decode 侧减速数据。

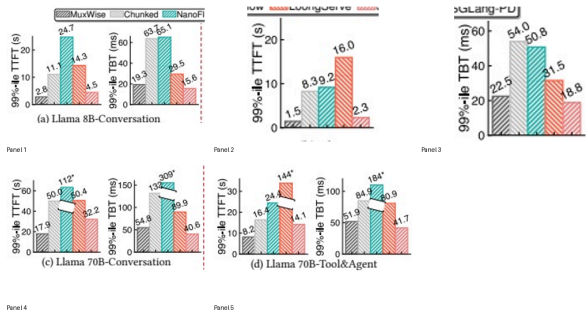


Figure 14. 99%-ile TTFT and TBT for Llama-8B and Llama-70B on real-world Conversation and Tool&Agent workloads. Chunked represents chunked-prefill in SGLang. Values marked with * are too large; we clip their corresponding bars, so the bar height only decodes their relative size.

图组解读 实验结论 呼应：第 9 节：具体主张

- **图组结论：** MuxWise 在多数真实负载下 99%-ile TTFT/TBT 普遍优于四基线，部分场景 SGLang-PD 的 TBT 更优。
- **论证关系：** 直接支撑端到端延迟优势主张，同时揭示与静态分离方案（SGLang-PD）在 TBT 上的权衡边界。
- **适用范围：** 覆盖 Llama-8B/70B 及 Conversation、Tool&Agent 负载。

表格证据：Table 1. Diverse patterns of typical LLM tasks. Minimum, mean, and maximum values for each metric are reported. **对比基线** 呼应：第 9 节：具体主张

- **图组结论：** 不同 LLM 任务在输入、输出及复用上下文长度上呈现出巨大的模式差异。
- **论证关系：** 量化真实工作负载的多样性与波动性，论证静态资源分配无法兼顾吞吐与 SLO 的根问题。
- **适用范围：** 统计涵盖 Single-turn, Long-doc, Reasoning, Multi-turn 等典型在线服务负载。

	Input length	Output length	Reused length
ShareGPT [4]	4/226/1024	4/195/1838	\
LooGLE [25]	3380/30k/81k	2/15/326	\
OpenThoughts [17]	311/709/4633	684/8374/32k	243
Conversation [34]	891/7538/123k	1/342/2000	0/4496/120k
Tool&agent [34]	891/8596/123k	1/182/2000	0/4905/120k

表格证据：Table 2. Compute analysis for prefill and decode phases. **对比基线** 呼应：第 9 节：具体主张

- **图组结论：** 给出 Prefill 与 Decode 阶段在 Attention 与 FFN 模块上的理论计算复杂度表达式。
- **论证关系：** 为抗争用估计器中的 solo-run predictor 线性模型拟合提供理论计算复杂度建模依据。
- **适用范围：** 适用于标准 Transformer 架构模型的 Prefill 与 Decode 计算量分析。

	Attention	FFN
Prefill w/o cache	$O(Ld^2 + L^2d)$	$O(Ld^2)$
Prefill w/ cache	$O(nd^2 + Lnd)$	$O(nd^2)$
Decode	$O(d^2 + (r + 1)d)$	$O(d^2)$

表格证据：Table 3. **实验结论** 呼应：第 9 节：具体主张

- **图组结论：** 补充展示 Llama-70B 在 Conversation 负载下各基线与 MuxWise 的吞吐与资源利用率等细节指标。
- **论证关系：** 与 Figure 14 延迟数据互补，全面支撑 MuxWise 综合性能优越性的主张。
- **适用范围：** 限定于 Llama-70B 在 Conversation 工作负载下的评估。

	TTFT (s)		TBT (ms)		E2E (s)		TPOT (ms)	
	Avg.	P50	Avg.	TBT	Avg.	P50	Avg.	P50
MuxWise	3.1	1.4	30.2	25.0	13.1	12.2	31.1	28.3
Chunked	12.0	7.2	45.3	49.3	27.0	23.3	46.9	45.7
NanoFlow	51.4	52.3	105.6	98.9	83.4	84.2	120.2	103.2
LoongServe	17.7	14.7	61.0	58.3	38.4	35.8	62.3	60.6
SGLang-PD	7.38	3.95	32.9	32.5	18.4	16.4	33.5	33.2

表格证据：Table 4. **实验结论** 呼应：第 9 节：具体主张

- **图组结论**：补充展示 Llama-70B 在 Tool&Agent 负载下各基线与 MuxWise 的吞吐与 SLO 达标情况等数据。
- **论证关系**：为 Tool&Agent 多轮工具调用场景下的端到端性能优势提供多维证据支撑。
- **适用范围**：限定于 Llama-70B 在 Tool&Agent 工作负载下的评估。

	TTFT (s)		TBT (ms)		E2E (s)		TPOT (ms)	
	Avg.	P50	Avg.	P50	Avg.	P50	Avg.	P50
MuxWise	1.3	1.0	27.2	24.1	4.0	1.6	33.1	27.7
Chunked	2.4	1.1	30.5	21.2	5.5	2.3	45.9	47.1
NanoFlow	2.8	1.2	58.8	42.4	7.4	2.5	70.3	62.6
LoongServe	59.9	56.0	52.4	50.8	65.2	61.0	56.2	54.6
SGLang-PD	2.1	1.5	31.6	31.0	5.2	2.3	37.1	33.7

表格证据：Table 5. Token throughput and GPU utilization for Llama8B and Llama-70B on Tool&Agent workload under Goodput. **实验结论** 呼应：第 9 节：具体主张

- **图组结论**：展示 SLO 约束（Goodput）下 Llama-8B/70B 在 Tool&Agent 上的 token 吞吐量与 GPU 利用率。
- **论证关系**：直接支撑论文关于 MuxWise 能在保 SLO 前提下显著提升有效吞吐与 GPU 利用率的主张。
- **适用范围**：覆盖 Llama-8B 与 Llama-70B 在 Tool&Agent 负载下的 Goodput 评测。

Model	Llama-8B		Llama-70B	
	Token/s	GPU Util.	Token/s	GPU Util.
MuxWise	25397	88.1	7430	84.0
Chunked	9768	63.8	2269	66.1
NanoFlow	4884	55.1	-	-
LoogServe	12698	75.3	2936	70.1
SGLang-PD	19535	P(72.4)/D(83.4)	4538	P(67.1)/D(81.9)

10. 完整因果推理树

- 根问题：LLM 服务需同时满足 prefill/decode 两条独立 SLO，二者交替执行致资源冲突
 - 旧方法限制：分离式部署资源耦合致闲置/重算；分块预填充存在预算两难
 - 核心洞见/假设：以 GPU 内空间并行替代时间交替，解耦 compute 与 memory 管理
 - 方法模块：无气泡多路复用引擎（层级化 prefill 切分+事件同步）
 - 可验证预测：层级调度可低开销消除气泡且不损害 goodput
 - 指标与实验：§ 4.4.2 消融（TTFT/TBT 变化量）[实验支持]
 - 图表证据：Figure 9 问题示意+解法示意
 - 结论与适用边界：气泡率轻微上升（7.7%vs4.5%）但作者称不影响 goodput, 该因果未被独立量化实验证明[基于文中逻辑的推断]
 - 方法模块：抗争用估计器（solo-run predictor+contention guard）
 - 可验证预测：预测误差可控、争用减速有上界
 - 指标与实验：预测误差数字（8.16%/8.84%）、Figure 11 争用画像[实验支持]
 - 图表证据：Figure 11 箱线图
 - 结论与适用边界：仅覆盖 decode 侧、离线画像覆盖范围外的线上争用是否超界未被验证[基于文中逻辑的推断]
 - 方法模块：SLO 感知调度器（分层启动量+抢占）
 - 可验证预测：抢占机制提升尾部 TTFT
 - 指标与实验：Figure 20 抢占消融（1.96×提升）[实验支持]
 - 图表证据：调度示意图
 - 结论与适用边界：关闭抢占仍有提升的说法未被量化验证[未被当前证据直接验证]
- 整体效果验证：端到端 TTFT/TBT/goodput 提升
 - 指标与实验：Figure 14-17 端到端对比[实验支持]
 - 图表证据：Figure 14 柱状图
 - 结论与适用边界：结论限定于已测试的模型/硬件/负载组合，Qwen-235B 场景仅单一基线对比[作者明确陈述]

11. 结论、贡献与边界

- **贡献总结**: 提炼高 goodput LLM 服务关键需求; 提出 PD 多路复用新范式及三模块设计; 在多样负载下评估并证明优于现有方案 [作者明确陈述]。
- **证据充分的结论**: 真实负载/合成负载下 MuxWise 相对四基线的 TTFT/goodput 提升有直接实验数字支撑 (§ 4.2-4.3) [实验支持]。
- **尚属推断的意义**: goodput 指标本身为推导关系而非原文公式, "气泡率略升不影响 goodput"这一因果解释缺乏独立对照实验[基于文中逻辑的推断]。
- **适用条件**: 仅在 SLO 严格的在线服务场景占优; 需硬件支持进程内低开销空间划分 (如 GreenContext) [作者明确陈述]。
- **局限**: 多数消融实验仅用 Llama-70B ("其他模型结果相似"未给量化数据) [作者明确陈述]; contention guard 的画像更新运维成本本文中未说明。
- **审稿式风险检查**:
 - ▶ 贡献新意: GreenContext 等底层技术非本文首创, 本文贡献在于系统整合与调度设计, 论文自身已区分[作者明确陈述], 当前证据未显示过度夸大风险。
 - ▶ 写作/可复现性: 代码/数据集链接文中未说明, 复现存在障碍, 属当前证据暴露的风险。
 - ▶ 效果大小: 主要提升数字来自论文自报的排除"不稳定结果"后的平均值, 统计口径 (是否多次运行、置信区间) 文中未说明, 构成一定风险。
 - ▶ 实验覆盖: MoE 模型场景仅对比单一基线, 覆盖面较窄, 属当前证据暴露的风险[作者明确陈述该限制]。
 - ▶ 方法设计: worst-case 估计依赖离线画像覆盖率, 在线场景是否超出画像范围未见实验验证, 当前证据未显示该风险已发生, 但也未排除。

12. 术语与第一性原理学习路径

12.1 核心术语

- **术语**: SLO (服务等级目标)。**第一性原理角色**: 连接"用户体验要求"与"系统资源分配"两个基本量的约束条件。**本文位置**: TTFT/TBT 两条 SLO 是全文优化目标。
- **术语**: KV 缓存。**第一性原理角色**: 受显存容量约束, 是 connect"历史 token 计算结果"与"当前生成 token"的因果媒介, 避免重复计算。**本文位置**: 共享显存设计的核心对象。
- **术语**: SM (流式多处理器)。**第一性原理角色**: GPU 计算资源的最小可分配单元, 受总数量物理约束。**本文位置**: 调度器分配对象、GreenContext 划分单位。
- **术语**: GreenContext。**第一性原理角色**: 连接"进程内多流"与"低开销 SM 划分"两个量的机制。**本文位置**: 本文实现空间复用的底层技术基础。
- **术语**: Goodput。**第一性原理角色**: 受 SLO 约束限定的吞吐量, 是"原始吞吐"与"SLO 合规率"两个量的耦合结果。**本文位置**: 全文核心评测指标。
- **术语**: Token budget (token 预算)。**第一性原理角色**: 受 GPU 算力约束与 TBT SLO 约束双重限制的资源分配参数。**本文位置**: chunked-prefill 两难的核心变量。
- **术语**: MoE (混合专家模型)。**第一性原理角色**: 受"总参数量"与"激活参数量"分离约束的架构, 降低计算成本。**本文位置**: Qwen3-235B 适用范围验证场景。

12.2 学习路径

1. 先理解 token 与 KV 缓存这两个基本对象——它们是 LLM 推理的最小计算/存储单元, 是理解 prefill/decode 差异的前提。
2. 再理解 SM 与显存这两类 GPU 资源约束——任何调度设计都受限于"有限 SM 数"与"有限显存容量"这两条硬约束。
3. 然后理解 prefill 计算密集、decode 显存带宽密集这一机制性差异——它解释了为何两阶段需要不同的资源分配策略。
4. 最后理解 TTFT/TBT/Goodput 这类可观测指标——它们是判断调度设计是否成功的最终标尺, 理解了它们才能读懂论文的实验图表。